

Trading Agent – Documentation (Assignment 1)

Vinnyk Yehor

1. Topic and goal of the agent

The agent is an autonomous trader on a simulated single-asset market. Its goal is to end a session of 8000 time steps (ticks) with the highest possible portfolio value, i.e. the largest profit over its initial budget.

At every tick the market produces a new price. The agent observes the price, the all-time high and low, its cash and the number of units it holds, and decides to **buy** one unit, **sell** one unit or **hold**. The price is driven by hidden market regimes (uptrend, downtrend, sideways), which the agent never sees directly.

The work compares two agents on identical markets:

- **TradingAgent** (own agent): a mean-reversion trader with internal memory. It buys when the price is unusually low compared with recent prices and sells only with a minimal profit.
- **BaseAgent** (baseline): a simple reflex agent that buys after every price drop and sells after every price rise.

The project is written in Python. The source code is in `src/`, the market settings in `tradingConfigs.json`, and all results in `results/`.

2. PEAS description

	Description	Where in code
P – Performance measure	Profit at the end of the session: final portfolio – initial budget, where final portfolio = cash + held units × last market price. Higher is better.	<code>expRunner.runExp</code>
E – Environment	A simulated market of one asset. The price follows geometric Brownian motion whose drift and volatility depend on a hidden regime (uptrend, downtrend, sideways); each regime lasts 50–150 ticks, then a new one is drawn by weight. A weak mean-reversion term pulls the price toward the starting price. The environment also holds the agent's cash, its holdings and the trade log.	<code>environment.TradingMarket,</code> <code>priceSimulator.PriceSimulator</code>
A – Actuators	Three actions: buy (one unit at the current price), sell (one unit at the current price), hold. A buy without enough cash or a sell without holdings is ignored.	<code>actuator.Actuator,</code> <code>TradingMarket.applyAction</code>
S – Sensors	The percept: current price, all-time high (ATH), all-time low (ATL), current cash (budget), number of held units. The regime, its remaining duration and future prices are not observable.	<code>sensor.Sensor,</code> <code>TradingMarket.getPerception</code>

The performance measure rates the outcome in the environment (money at the end), not the way the agent reaches it. It does not reward buying, selling or trading often. An agent that never trades scores exactly 0.

3. Environment classification

The environment is partially observable, stochastic, sequential, dynamic, discrete and single-agent.

Property	Classification	Justification
Observability	Partially observable	The agent sees the price, ATH, ATL, cash and holdings, but not the current regime, how long it will last, the drift and volatility parameters or future prices. It has to estimate the market state from the price history.
Determinism	Stochastic	Each new price contains a random Gaussian shock, and the next regime and its length are drawn at random. The same action in the same state can lead to different future outcomes. The agent's own actions are deterministic: an allowed buy or sell always executes at the current price.
Episodic / sequential	Sequential	Every decision changes the agent's future state and options: a buy reduces cash and increases holdings, which limits later buys and enables later sells. The trading agent also stores reference prices that shape its later decisions.
Static / dynamic	Dynamic (in time steps)	The price changes every tick regardless of what the agent does, and the regime switches over time. The agent acts once per tick; time is not stopped for it.
Discrete / continuous	Discrete time and actions, continuous price	Time runs in integer ticks and there are exactly three actions. The price and the cash are real numbers.
Agents	Single-agent	Only one agent trades. Its trades do not move the price: the market is a simulation, not an order book.

The assignment requires a sequential environment plus at least one extra property. This environment has four: **partial observability** (hidden regime), **stochastic outcomes** (random

prices), **dynamic changes** (regime switches) and **limited resources** (a fixed budget of 800, so the agent cannot buy at every low price).

4. Representation of the environment, percepts and actions

The system has five separate parts. The agent never touches the environment's internal state: it receives a percept from the sensor and returns an action string to the actuator.

Part	Class / function	Role
Environment	<code>TradingMarket</code> (<code>environment.py</code>)	Holds the true state and applies actions
Price generator	<code>PriceSimulator</code> (<code>priceSimulator.py</code>)	Produces the next price; holds the hidden regime
Sensor	<code>Sensor.sense()</code> (<code>sensor.py</code>)	Returns the percept
Agent	<code>TradingAgent / BaseAgent</code> (<code>agent.py</code> , <code>agent2.py</code>)	<code>makeDecision(percept)</code> returns an action
Actuator	<code>Actuator.act(action)</code> (<code>actuator.py</code>)	Passes the action to the environment
Performance measure	<code>profit in runExp</code> (<code>expRunner.py</code>)	Rates the session at its end

Environment state

`TradingMarket` stores: current price, price history (one value per tick), ATH and ATL, cash (`_budget`), held units (`_holdings`), the tick counter and the trade log, a list of (`tick`, `action`, `price`) for every executed trade.

`PriceSimulator` stores the hidden part: the current regime, the ticks left in it, the regime weights and its own random number generator `random.Random(seed)`. Each regime has its own drift μ and volatility σ :

Regime	μ (drift per tick)	σ (volatility per tick)
uptrend	+0.0006	0.010
downtrend	-0.0006	0.010
sideway	0	0.006

One price step is a geometric Brownian motion log-return with mean reversion toward the starting price ($\theta = 0.0002$, Z a standard normal random number):

$$\ln \frac{P_{t+1}}{P_t} = \left(\mu - \frac{1}{2} \sigma^2 \right) + \theta \frac{P_0 - P_t}{P_0} + \sigma Z$$

The price is never allowed to fall below 0.01. When a regime runs out, the next one is drawn with `random.choices` using the configured weights, and its length is drawn uniformly from 50 to 150 ticks.

Percept

A Python dictionary returned by `sensor.sense()`:

```
{"price": 142.37, "ath": 573, "atl": 78, "budget": 657.6, "holdings": 1}
```

ATH and ATL start from the values in `tradingConfigs.json` (573 and 78) and are widened whenever the price breaks them.

Actions

An action is one of three strings:

Action	Effect	When it is ignored
"buy"	holdings + 1, cash – price, logged	cash < price
"sell"	holdings – 1, cash + price, logged	holdings = 0
"hold"	nothing	–

One time step

1. `env.tick()` generates the new price and updates ATH and ATL.
2. `sensor.sense()` returns the percept.
3. `agent.makeDecision(percept)` returns an action.
4. `actuator.act(action)` executes it in the environment.

This repeats for 8000 ticks. Then the performance measure is computed and the run is logged.

5. Performance measure

The performance measure is the profit at the end of a session of fixed length (8000 ticks):

$$\text{profit} = \text{cash}_T + \text{holdings}_T \cdot P_T - \text{initial budget}$$

T is the last tick and P_T the last market price. Units still held at the end are valued at that price, so an agent cannot hide a loss by refusing to sell.

Why this measure is suitable:

- **It rates the result, not the method.** It does not reward any particular action, the number of trades or holding a position. Two agents with different strategies are compared by the money they end with.
- **It penalises cost and bad timing.** Buying high and selling low lowers cash; buying too early leaves the agent without cash for better prices later.
- **It is the same for both agents.** Same market (same seed), same budget, same horizon.
- **It has a neutral point.** Doing nothing scores 0, so a negative profit means the strategy destroyed value.

The fixed horizon matters. An earlier version kept the session running until the agent had sold everything; the trading agent then waited thousands of extra ticks for prices to recover, which inflated its profit and never ended in a downtrend. Valuing the open position at the last price removes this bias.

For each run the logger also records the number of ticks, the run time, final cash, final holdings and the number of buys and sells. These are diagnostics for the interpretation, not part of the measure.

6. Base agent

BaseAgent (agent2.py) is a simple reflex agent. It reacts only to the last price change and has no notion of trend, volatility or profit.

Its only memory is the previous price. Its rule at each tick:

1. On the first tick, store the price and hold.
2. If the price **fell** since the last tick and cash $\geq$ price $\rightarrow$ buy.
3. Else if the price **rose** since the last tick and holdings $> 0 \rightarrow$ sell.
4. Otherwise $\rightarrow$ hold.
5. Store the current price as the previous price.

This is the natural "buy low, sell high" reflex applied to every single tick. It is a fair baseline because it uses the same percept and the same actions as the own agent, under identical conditions. Because random price noise makes the price go up or down almost every tick, it trades nearly constantly: about 2900–3400 buys and as many sells per session (section 11).

7. Own agent: TradingAgent

`TradingAgent` (`agent.py`) is a mean-reversion trader. It buys only when the price is unusually low compared with the recent past, adds to a position only at clearly lower prices, and sells only at a profit.

Idea

The market regime is hidden, so the agent cannot know whether the market is rising or falling. Instead it estimates the "normal" price from the last `windowLen` prices: their mean μ and standard deviation σ . A price far below μ is likely a temporary dip caused by noise, and the environment's mean reversion makes a return toward normal more likely than a further fall. Such a dip is the **demand zone**.

Parameters

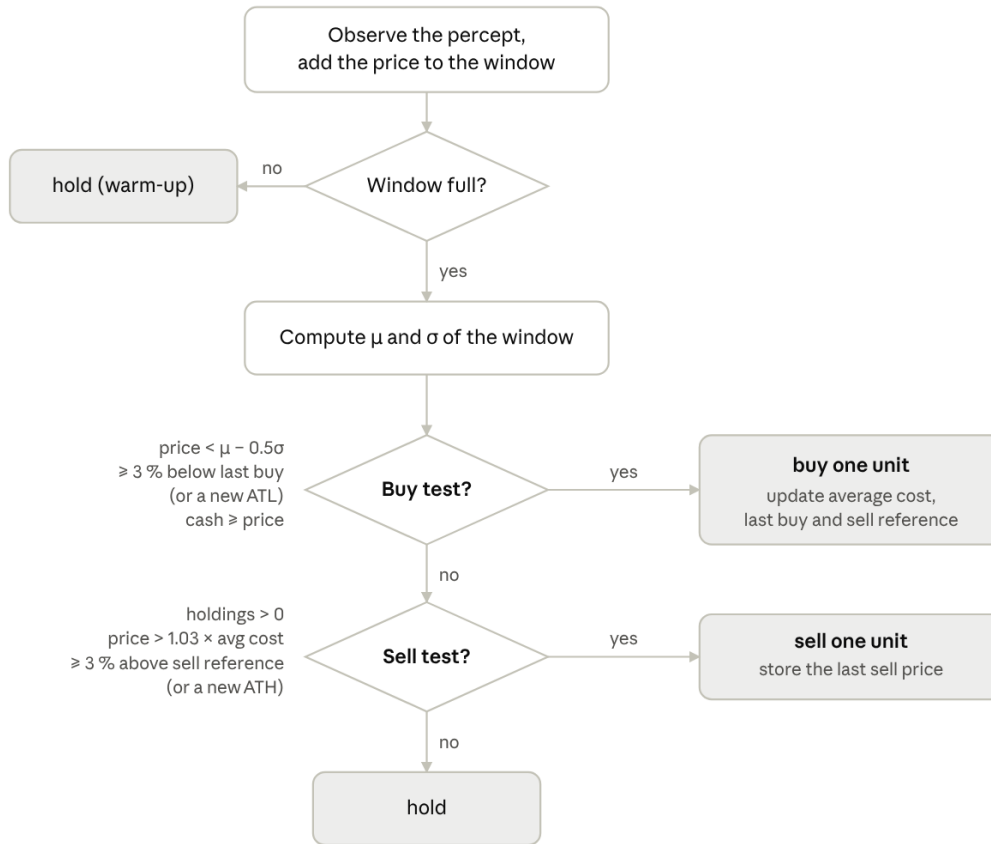
Parameter	Value	Meaning
<code>windowLen</code>	Half of regime duration ticks	length of the price memory; set to $(150 - 50) / 2$, the half of regime length, so the window covers roughly half of regime
<code>k</code>	0.5	how many standard deviations below the mean counts as cheap
<code>minMargin</code>	0.03	minimal relative profit (3 %) for a sell, and minimal relative step between consecutive buys or sells

Decision procedure (each tick)

1. **Observe and remember.** Append the price to the window (the oldest price drops out).
2. **Reset when flat.** If holdings = 0, forget the reference prices of the previous position.

3. **Warm-up.** If the window holds fewer than 50 prices → hold. Without enough history, μ and σ are unreliable.
4. **Estimate the market.** Compute μ = mean of the window and σ = population standard deviation.
5. **Demand zone test.** The price is in the demand zone if $\sigma > 0$ and price $< \mu - k \cdot \sigma$.
6. **Better price to buy (averaging down).** True if this is the first buy of a position, or the price is at least 3 % below the last buy price, or the price is a new all-time low.
7. **Buy** if the price is in the demand zone, it is a better price to buy, and cash $\geq$ price. Then update the average buy price of the position, store the last buy price, and set the next sell reference to average buy price $\times 1.03$.
8. **Better price to sell (scaling out).** True if the price is at least 3 % above the sell reference, or the price is a new all-time high. The sell reference is the last sell price; right after a buy it is the average buy price $\times 1.03$.
9. **Sell** if holdings > 0 , the price is more than 3 % above the average buy price, and it is a better price to sell. Store this price as the last sell price. Because steps 8 and 9 combine, the first sell of a position needs about 6.1 % profit (1.03×1.03) unless the price sets a new ATH; in the experiments every sell was 6.1–6.8 % above the average buy price.
10. Otherwise → hold.

The agent trades only when every condition holds; otherwise it holds



TradingAgent.makeDecision · one tick, 3 decisions

The buy test is checked before the sell test, so in one tick the agent either buys, sells or holds, never both.

The buy and sell conditions as formulas:

$$\text{buy} \Leftrightarrow P_t < \mu - k\sigma \wedge (P_t < 0.97 P_{\text{lastBuy}} \vee P_t \leq \text{ATL} \vee \text{first buy}) \wedge \text{cash} \geq P_t$$

$$\text{sell} \Leftrightarrow \text{holdings} > 0 \wedge P_t > 1.03 \bar{P}_{\text{buy}} \wedge (P_t > 1.03 P_{\text{sellRef}} \vee P_t \geq \text{ATH})$$

The 3 % gaps between consecutive buys and between consecutive sells stop the agent from spending its whole budget, or selling its whole position, during one short dip or spike. They spread the trades over a larger price move.

Why this is more than a reflex

The base agent reacts to one price change. The trading agent decides from a model of the market built from 50 observations (μ , σ), plus a model of its own position (average cost, last buy, last sell). Its action therefore depends on the whole recent history and on what it did before, not only on the last percept.

8. Internal state and memory

The trading agent keeps four pieces of memory between ticks. None of them is visible in the percept; the agent builds them from what it has observed and done.

State variable	Type	Updated when	Used for
<code>_window</code>	<code>deque(maxlen=100)</code> of prices	every tick (oldest price drops out)	estimating the normal price μ and the volatility σ
<code>_averageBuyPrice</code>	number	every buy: $(\text{holdings} \cdot \text{old average} + \text{price}) / (\text{holdings} + 1)$	the profit threshold for selling
<code>_lastBuyPrice</code>	number	every buy	requiring the next buy to be $\geq 3\%$ cheaper
<code>_lastSellPrice</code>	number	every buy (set to $\text{average} \times 1.03$) and every sell	requiring the next sell to be $\geq 3\%$ higher

When holdings drop to 0, the three position variables are reset to 0, so a new position starts with fresh reference prices. The window is never reset: market knowledge carries over between positions.

The base agent's whole memory is one number, the previous price.

The internal state is what lets the trading agent act well under partial observability. The window replaces the missing regime information with a statistical estimate. The position variables replace a full trade history with the three numbers the decision actually needs.

9. Rationality, autonomy and limits

The agent is rational in the bounded sense: given only what it can observe and remember, it picks the action expected to raise the final portfolio value. It is not omniscient, so a rational action can still lose money.

What "doing the right thing" means here

The right action is the one that maximises the expected final portfolio value at tick 8000. In practice: buy when a unit is cheap relative to its likely future value, sell when the price gives a real profit, and keep enough cash to use later opportunities. Trading for its own sake is not the goal; every trade must move the agent toward more money at the end.

What the agent bases its action on

- the current percept: price, ATH, ATL, cash, holdings;
- its estimate of the market from the last 100 prices (μ , σ);
- its position memory: average buy price, last buy price, last sell price.

Known in advance (built into the agent)

- the three possible actions and that a trade moves exactly one unit;
- the parameters $k = 0.5$ and $\text{minMargin} = 3\%$;
- the window length of 100 ticks, derived from the configured regime length (50–150 ticks). This is prior knowledge about the environment's structure;
- the starting ATH = 573 and ATL = 78 from the configuration;
- the implicit assumption that prices tend to return toward their recent mean.

The agent does **not** know: the current regime, the regime weights, μ and σ of the regimes, the mean-reversion strength, the seed or any future price.

Learned only during operation

- every price, and through the window the current normal price and volatility;
- new ATH and ATL values;
- its own cash and holdings after each trade (through the percept);
- the cost of its current position.

Use of past experience

Within a session, yes: the window is a short-term memory of the market, and the position variables remember its own past trades, so the same percept can lead to different actions depending on history. Between sessions, no: each run starts with an

empty window and no learning carries over. The parameters k , `minMargin` and `windowLen` are set, not learned.

Autonomy

During a session the agent acts without any human input. Its behaviour adapts to what it observes: the demand zone moves with the market because μ and σ are recomputed every tick. Its autonomy is partial: the decision rules and the parameters come from the designer, and it cannot change them from experience.

Bounded rationality

- **Limited information.** It does not see the regime, so it cannot tell a temporary dip from the start of a long downtrend.
- **Simple model.** It uses only the mean and standard deviation of 100 prices. Trends inside the window, regime changes and longer history are ignored.
- **Fixed rules and parameters.** k , the 3 % margin and the window length are the same in every market; they are not tuned to the current regime.
- **One unit per tick.** It cannot scale a trade with its confidence.
- **Limited budget.** 800 at a starting price of 140 buys at most about 5–6 units at normal prices, so cash spent too early is missing later.
- **Limited computation.** It does $O(\text{window})$ work per tick (about 0.1 ms), with no lookahead or planning over future scenarios.

Why a good decision does not always lead to success

The market is stochastic and only partially observable. A buy in the demand zone is a good bet because a low price is more likely to recover than to fall further. But if the hidden regime has just switched to a downtrend, the price keeps falling, the agent buys again 3 % lower, and it can run out of cash while holding units worth less than it paid. The decision was reasonable with the information available; the outcome was bad because of information the agent could not have.

For this reason the agent must be judged over several market configurations and by average results, not by one successful run. Sections 11 and 12 show both cases: the same rules earn +812 in one market and lose −97 in another.

10. Experiments and reproducible setup

Both agents were run on four market configurations of increasing friendliness, with identical settings and the same seed, so each pair of runs sees exactly the same price path.

Current settings (`tradingConfigs.json`)

Setting	Value
starting price	140
initial ATH / ATL	573 / 78
regime length	50–150 ticks (uniform)
initial budget	800
seed	44
session length	8000 ticks (<code>expRunner.runExp</code>)
trading agent	<code>windowLen = 100, k = 0.5, minMargin = 0.03</code>

Configurations

The configurations differ only in the regime weights, which set how often each hidden regime is drawn. This changes how hard the problem is: the more often downtrends occur, the more often a dip is the start of a real fall rather than noise.

Config	Weights up : down : side	Probability of a downtrend	Market type
config0	1 : 1 : 1	33 %	balanced, hardest
config1	1 : 1 : 3	20 %	mostly sideways
config2	2 : 1 : 3	17 %	mild uptrend
config3	4 : 1 : 1	17 %	strong uptrend, easiest

Reproducibility

- All randomness comes from one `random.Random(seed)` inside `PriceSimulator`; the agents use no randomness. The same seed and settings always produce the same prices, trades and profits.
- Re-running `python src/runExperiments.py` reproduced every value in section 11 exactly.

- To test another market, change only `seed` in `tradingConfigs.json`. Other parameters: budget, prices and regime lengths in `tradingConfigs.json`; weights in `runExperiments.py`; `k` and ticks in `expRunner.py`.

What is recorded

Each run is logged by `resultLogger.py`:

- `results/runs.csv`: one row per run with config, agent, seed, ticks, run time in seconds, profit, final portfolio, final cash, final holdings, number of buys and sells;
- `results/trades/config{i}-{agent}.csv`: the course of the run, every executed trade as tick, action, price;
- `results/graphics/`: a price chart of every run with buy and sell markers, and the comparison chart.

Every run lasts exactly 8000 ticks. The steps needed for the result are therefore fixed; the run time is about 0.3–0.9 s for the trading agent and 0.005–0.02 s for the base agent on a laptop.

11. Results

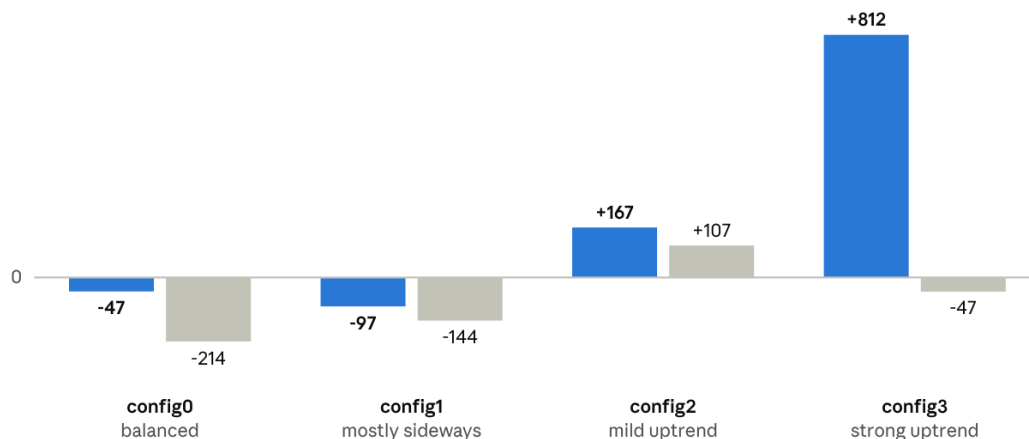
The trading agent earned more than the base agent in every configuration, by 47 to 859. Across the four configurations it averaged **+209 (standard deviation 418)**, the base agent **-75 (standard deviation 139)**. Both lost money in the two hardest markets.

Trading agent beats the base agent in all four markets

Profit after 8000 ticks (initial budget 800, seed 44)

■ Trading agent

■ Base agent



results/runs.csv · 8 runs, seed 44

Run diagnostics

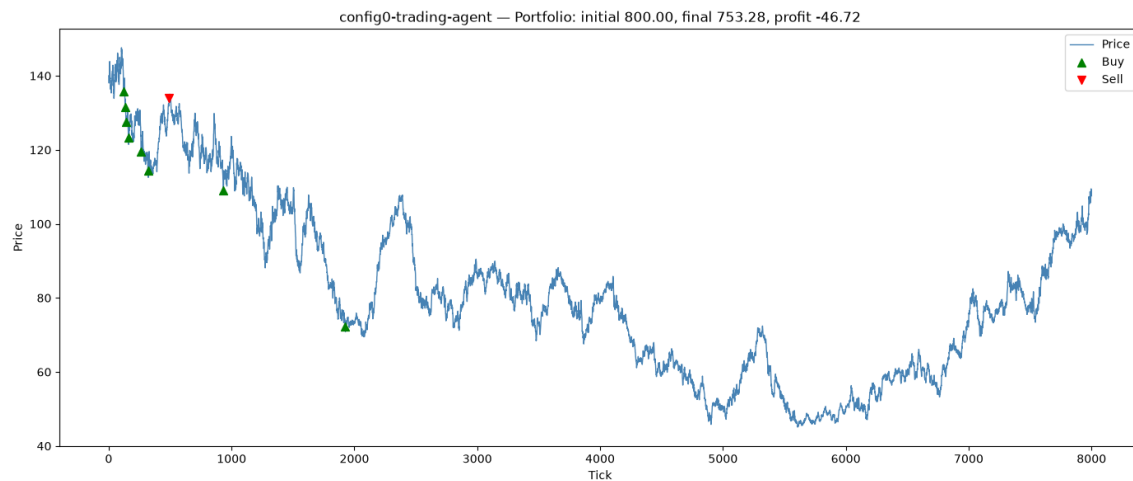
Config	Agent	Cash at end	Units held at end	Buys	Sells	Ticks	Run time (s)
config0	Trading	0.26	7	8	1	8000	0.89
config0	Base	155.47	4	3400	3396	8000	0.02
config1	Trading	0.40	6	8	2	8000	0.85
config1	Base	304.37	3	3367	3364	8000	0.01
config2	Trading	967.44	0	9	9	8000	0.72
config2	Base	352.60	3	3360	3357	8000	0.01
config3	Trading	711.34	3	35	32	8000	0.79
config3	Base	152.22	2	2942	2940	8000	0.02

The price path of each run, from the same seed:

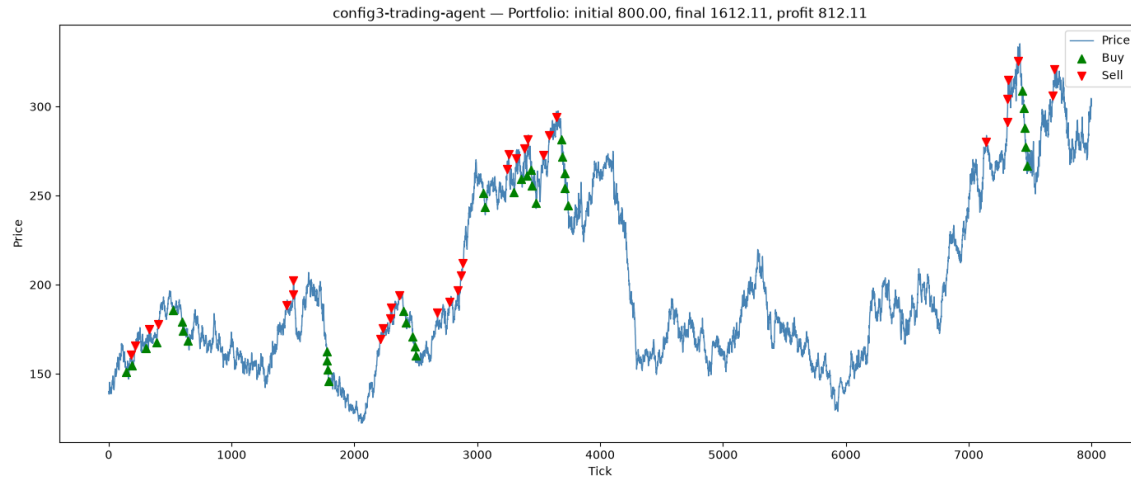
Config	Start price	End price	Lowest	Highest
config0	140	107.6	45.1	147.6
config1	140	117.1	64.0	163.8
config2	140	184.8	74.1	186.6
config3	140	300.3	122.3	335.3

Price charts of the trading agent

The hardest market (config0): 8 buys during the first 2000 ticks, 1 sell, then no trade for about 6000 ticks.



The easiest market (config3): 35 buys and 32 sells spread over the whole session.



All eight price charts are in `results/graphics/`, every trade in `results/trades/`.

12. Comparison and interpretation

The trading agent wins because it trades rarely and only at statistically low prices; it fails when a dip turns into a long fall, because it spends its whole budget early and then cannot act for thousands of ticks.

Why the agents behave differently

The base agent trades on noise. Almost every tick the price moves up or down by chance, so the base agent buys or sells almost every tick: about 3000–3400 trades of each kind per run. A single drop says nothing about the trend, so its decisions are close to random with respect to where the market is going. It sells after every rise, so it holds on average only 1.5–3 units (at most 7) and gains little from a rising market: in config3 the price more than doubled (140 → 300) and the base agent still lost 47. In falling stretches it keeps buying on every down-tick and holds units that lose value, which is why its worst result is config0 (−214).

The trading agent trades on a model. It acts only when the price is at least 0.5σ below the mean of the last 100 prices, and sells only about 6 % above its average cost. It therefore made only 9–67 trades per run. In markets where dips recover, almost every position is closed at a profit:

- **config2 (mild uptrend):** 9 buys and 9 sells, every position closed, ended with 967 in cash and nothing held. Profit +167, all of it realised.

- **config3 (strong uptrend):** 35 buys and 32 sells. Dips inside the uptrend recovered quickly, so the agent completed many profitable round trips. It ended with 711 in cash plus 3 units bought at an average 288 while the price was 300. Profit +812.

Where the own agent failed

In config0 and config1 the trading agent lost money (−47 and −97). It still did better than the base agent, but it did not reach its goal. The cause is the same in both runs:

1. Early in the session the price started a long decline (config0: from 140 to about 72 by tick 1925).
2. Each new low passed the demand-zone test and was more than 3 % below the last buy, so the agent kept averaging down: 8 buys in each run.
3. The budget ran out. Cash at the end was 0.26 (config0) and 0.40 (config1); the last trade happened at tick 1925 and tick 1026.
4. To sell, the price had to rise about 6 % above the average cost of the held units: about 122.5 in config0 (7 units at 115.5) and about 145.7 in config1 (6 units at 137.3). After the last trade the price never got above 109.5 and 119.1.
5. The agent was frozen for 6000–7000 ticks. It could not sell, and it could not buy at the real bottom (45.1 in config0) because it had no cash.

Each buy was a reasonable decision with the information available: the price really was far below its recent mean. The agent could not see that the regime had become a downtrend, and it has no rule for cutting a losing position. This is the "good decision, bad outcome" case from section 9: the strategy is sound for noise around a stable level and weak against a persistent trend.

Variability

The trading agent's results vary much more across markets: standard deviation 418 against 139 for the base agent. Its outcome depends strongly on whether the market recovers after it has bought, while the base agent's churning keeps its results in a narrow band near or below zero. Higher average profit comes with higher risk.

Each configuration was run with one seed, so each bar in section 11 is a single sample. A different seed gives a different price path and different profits; the conclusions above describe the mechanisms seen in these runs rather than precise averages.

Steps and time

Every run lasted 8000 ticks. The trading agent needs about 0.1 ms per tick (mean and standard deviation over 100 prices), the base agent about 2 μ s per tick (one comparison). Both are far below any practical limit on decision time.

13. Shortcomings and possible improvements

The main weakness is budget lock-up in falling markets; the main gap in the evaluation is one seed per configuration.

Shortcoming	Effect seen	Possible improvement
No exit from a losing position (no stop-loss)	config0 and config1: frozen for 6000–7000 ticks with units bought too high	sell when the price falls a set percentage below the average cost, or after a maximum holding time
Averaging down in 3 % steps spends the budget during one fall	8 buys and cash < 1 within the first 2000 ticks	larger steps between buys (e.g. 1 σ or 10 %), or a cap on the share of the budget in one position
No trend detection	dips in a downtrend look like dips in a sideways market	compare a short and a long moving average, or the slope of the window, and stop buying while the trend points down
Sell threshold is effectively about 6.1 %, not the intended 3 % (the reference is set to average $\times$ 1.03 at a buy, then a sell needs another 3 %)	positions are held longer than intended	set the sell reference to the average buy price itself, so the first sell needs 3 %
Fixed parameters ($k = 0.5$, 3 %, window 100)	the same rules in every market	tune them on training seeds, or adapt k to the measured volatility
One unit per trade	cannot scale a trade with how far the price is below the mean	trade size proportional to the distance from μ in units of σ
No learning between sessions	every run starts from zero knowledge	keep the estimated parameters, or learn the thresholds from past runs
Evaluation: one seed per configuration	each result is a single sample; the standard deviation in section 11 is across markets, not within a market	run each configuration with 20 or more seeds and report mean and standard deviation per configuration
Comparison chart averages different markets	<code>Comparison.png</code> mixes easy and hard markets into one bar	plot one bar pair per configuration, as in section 11

14. User guide

Install matplotlib, then run `python src/runExperiments.py` from the project root; the results appear in `results/`.

Requirements

- Python 3.10 or newer (developed with Python 3.14)
- `matplotlib` (the only external library, listed in `requirements.txt`)

Installation

`cd Project-1`

```
python3 -m venv .venv          # optional
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Running

Start the program from the project root, the folder that contains `tradingConfigs.json`. All paths are relative to this folder.

`cd Project-1`

```
python src/runExperiments.py
```

In PyCharm set *Run* → *Edit Configurations...* → *Working directory* to the `Project-1` folder, not `src/`. The folders `results/trades/` and `results/graphics/` must exist; both are included in the project.

The program prints nothing. It runs both agents on the four configurations and overwrites the files in `results/` (section 10 lists them).

Changing parameters

What	Where
seed, starting price, ATH/ATL, regime lengths, budget	<code>tradingConfigs.json</code>
regime weights of the four configurations	<code>if __name__ == "__main__":</code> block of <code>src/runExperiments.py</code>
session length (8000 ticks)	<code>ticks</code> parameter of <code>runExp</code> in <code>src/expRunner.py</code>
<code>k</code> of the trading agent (0.5)	<code>runExpTradingAgent</code> in <code>src/expRunner.py</code>

What	Where
<code>minMargin (0.03)</code>	default of <code>TradingAgent.__init__</code> in <code>src/agent.py</code>

Example: set `"seed": 7` in `tradingConfigs.json`, run the program again, and compare `results/runs.csv` and the charts with the previous run.

Project files

File	Content
<code>src/runExperiments.py</code>	entry point; loads the settings, runs all experiments, draws the comparison
<code>src/expRunner.py</code>	one session: sense → decide → act loop, profit, logging, chart
<code>src/environment.py</code>	<code>TradingMarket</code> , the environment
<code>src/priceSimulator.py</code>	<code>PriceSimulator</code> , prices with hidden regimes
<code>src/sensor.py</code> , <code>src/actuator.py</code>	<code>Sensor</code> , <code>Actuator</code>
<code>src/agent.py</code> , <code>src/agent2.py</code>	<code>TradingAgent</code> (own agent), <code>BaseAgent</code> (baseline)
<code>src/resultLogger.py</code>	writes <code>runs.csv</code> and the trade logs
<code>src/resultVizualization.py</code> , <code>src/compareAgentsVisualization.py</code>	price chart of one run, comparison chart
<code>tradingConfigs.json</code>	market settings
<code>README.md</code> , <code>requirements.txt</code>	short guide, dependencies

Libraries and external sources

Library or source	Used for
Python standard library: <code>random</code> , <code>math</code> , <code>statistics</code> , <code>collections</code> , <code>json</code> , <code>csv</code> , <code>os</code> , <code>time</code> , <code>operator</code>	random numbers, price formula, mean and standard deviation, the price window, configuration, logging, timing
<code>matplotlib</code>	charts only
Geometric Brownian motion	standard model of asset prices, used as the basis of the price generator